

Authoritative Deep-Research Report for `vi-history-suite` (Unresolved Workstreams Only)

Generation date (America/Hermosillo): 2026-04-02

Executive summary

`vi-history-suite` already implements the baseline “VI History” eligibility logic, VI magic-byte detection, Git-backed history viewer webview, and menu gating. The remaining engineering risk concentrates in five unresolved workstreams: (1) generating LabVIEW comparison reports from two Git revisions, (2) safely storing and rendering those reports inside a VS Code webview, (3) detecting LabVIEW 2026 Q1 tooling and selecting 32-bit vs 64-bit correctly across platforms, (4) progress UX plus workspace-trust gating for external tool execution and trust-sensitive settings, and (5) packaging/testing/CI mechanics for a VS Code extension that runs on desktop/remote-host.

The key authoritative finding is that **LabVIEW CLI’s `CreateComparisonReport` operation is explicitly designed to programmatically compare two VIs and output an HTML/XML/Word/text report**, including `HTMLSingleFile` for embedded assets, while `LVCompare` **is explicitly described as an interactive tool equivalent to the “Compare VIs” dialog** and has documented limitations around same-name VIs. The recommended architecture is therefore:

- Use **LabVIEW CLI `CreateComparisonReport` as the primary report engine** when available, strongly preferring `HTMLSingleFile` to avoid multi-file webview resource handling.
- Treat `LVCompare` as an **interactive fallback** only, and only if your UX can tolerate non-headless behavior and same-name limitations.
- Store reports under `context.storageUri` (workspace-scoped, extension-private storage) and expose them into a webview using `asWebviewUri` + **tight** `localResourceRoots` + CSP patterns.
- Declare workspace trust via `capabilities.untrustedWorkspaces` and gate *all* external process execution (LabVIEWCLI/LVCompare) plus trust-sensitive configuration keys using `restrictedConfigurations`.

Finally, the repo inspection indicates the codebase is ready to add a “report generation subsystem” as isolated modules (locator + engine + report store + UI integration) without disrupting existing history/indexing logic.

Repo-state assumptions you are making

Observed repo state (from inspecting the provided zip)

- The extension already has a working activation endpoint at `src/extension.ts`, a VI eligibility indexer at `src/indexing/viEligibilityIndexer.ts`, Git CLI adapter at

`src/git/gitCli.ts`, VI signature detection at `src/vi/viSignature.ts`, and a webview history panel at `src/history/viewer/*`.

- The manifest (`package.json`) already contributes a context menu command “**VI History**” in the Explorer menu and gates visibility by checking membership in a context-set (`resourcePath` in `viHistorySuite.eligiblePaths`).
- The extension currently sets a custom context key `viHistorySuite.isWorkspaceTrusted` at activation time, rather than using VS Code’s built-in `isWorkspaceTrusted` context key.
- There is **no implemented report generation module** (no code locating LabVIEWCLI/LVCompare, no report storage under `context.storageUri`, no webview embedding of generated HTML reports).

Explicit additional assumptions (not confirmed in repo)

- The extension will generate **two** report “types” consistent with the naming scheme `{type}-report-{fullFilename}.html` (`diff-report-*` and optionally `print-report-*`). If only one report type is desired, the scheme still applies with `type = diff`.
- The extension is **desktop/remote-host only** (not VS Code Web). This matters because running external executables is not possible in web extensions, and storage access differs in web extensions. For web extension restrictions, see the official “Web Extensions” guide.
- The product intends to support LabVIEW comparison/report generation primarily on **Windows and Linux** (macOS report generation is treated as contingent on tool availability and installed edition).

Decision matrix for unresolved workstreams

Workstream	Options considered	Recommended decision	Why (source-backed)	Key risk
Comparison report engine	(A) LabVIEWCLI <code>CreateComparisonReport</code> (B) <code>LVCompare</code> CLI (interactive)	A primary , B fallback	<code>CreateComparisonReport</code> is explicitly documented to <i>programmatically compare two VIs and output HTML/XML/Word/text</i> , including <code>HTMLSingleFile</code> embedded assets. <code>LVCompare</code> is documented as equivalent to interactive “Compare VIs” dialog.	If <code>CreateC</code> or CLI is not configured, different U
Handling same-name VI constraints	(A) Compare in-place (B) Copy both revisions to temp with unique filenames	B	LabVIEW cannot load two VIs with the same name; renaming is required for comparison.	Requires n naming an
Report storage	(A) <code>context.storageUri</code> (workspace-specific) (B) workspace folder (C) OS temp	A	VS Code API defines <code>storageUri</code> specifically for workspace-scoped extension-private storage; directory might not exist and must be created.	Cleanup and managem
Webview integration	(A) Render report HTML in webview via <code>asWebviewUri</code> (B) Open report in external browser	A	Webviews can load local content only through <code>asWebviewUri</code> and must restrict local access using <code>localResourceRoots</code> + CSP.	CSP + relat if report is

Workstream	Options considered	Recommended decision	Why (source-backed)	Key risk
LabVIEW runtime/ tool detection	(A) User config only (B) Scan standard install paths (C) registry probing + scan	C (hybrid)	NI docs provide standard install roots for LabVIEW and LVCompare paths; CLI supports <code>-LabVIEWPath</code> selection. Registry is not fully documented, so treat as best-effort.	Installed p edition co (LVCompa docs).
Workspace trust + restricted settings	(A) No manifest declaration (B) <code>supported:false</code> (C) <code>supported:'limited'</code> + restricted configurations	C	Workspace Trust guide defines <code>capabilities.untrustedWorkspaces</code> and <code>restrictedConfigurations</code> for settings used in trust-sensitive execution flows.	Complexity functional Mode is co
Progress UX	(A) Only <code>window.withProgress</code> (B) + status bar item; (C) + webview progress	B + C	Progress API supports multiple locations; status bar items are a standard extension surface; webviews support message passing for live progress updates.	Avoid nois progress a cancellatio
Packaging/ testing/CI	(A) manual vsix only (B) <code>vsce package</code> + <code>@vscode/test-electron</code> + CI publish pipeline	B	Official publishing docs specify <code>vsce package/publish</code> and recommended CI patterns; official testing docs specify <code>@vscode/test-cli</code> + <code>@vscode/test-electron</code> .	Tooling av runners, h dependen

Implementation guidance by workstream

Comparison report generation

Source-backed facts

1. **LabVIEW CLI** `CreateComparisonReport` **is explicitly designed to programmatically compare two VIs and generate reports.** NI documents that it “compare[s] two specified VIs” and outputs “an HTML, XML, Word, or text report” and is intended to “compare VIs without including VI dependencies,” helping avoid missing-dependency errors.
Source: NI “Comparing VIs using the LabVIEW Command Line Interface” (`CreateComparisonReport` operation) — <https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>
 2. **Supported report types and their asset behavior are explicitly documented:**
 3. `HTML` produces HTML with external images and a style file.
 4. `HTMLSingleFile` produces HTML with embedded images and style definition.
 5. `MicrosoftWord` requires Word installed.
 6. `PlainText` and `XML` exist (XML with external images).
- Source: same NI `CreateComparisonReport` doc — <https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>

7. CLI argument behavior is explicit:

8. If `-reportType` is not specified, it is inferred from `-reportPath`; the default is `HTMLSingleFile`.
9. `-o` overwrites existing reports and supporting files in the report directory.
10. `-c` creates a directory if it does not exist.
Source: NI CreateComparisonReport doc — <https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>
11. **LabVIEW cannot load/compare two VIs with the same name in the same context.** NI states “LabVIEW cannot load two VIs with the same name” and you must rename one to compare, and that LVCompare also uses the Compare VIs dialog.
Source: NI “Comparing VIs” — <https://www.ni.com/docs/en-US/bundle/labview/page/comparing-vis.html>
12. `LVCompare` is documented as interactive and has same-name limitations.
13. NI describes LVCompare as equivalent to the “Compare VIs” dialog and “compare differences between two VIs interactively,” and gives the LVCompare command line syntax with optional `-lpath` and feature-exclusion flags.
Source: NI “Configuring Source Control with LVCompare.exe” — <https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvcompareexe.html>
14. NI’s support article explicitly notes: “Comparing VIs with the same name is not supported by the LVCompare.exe utility.”
Source: NI KnowledgeBase “Compare Two VIs in LabVIEW” — <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000000YHwACAW&l=en-US>
15. **LabVIEW CLI operational prerequisites and behavior:**
16. NI states all CLI arguments are case sensitive.
17. CLI requires VI Server TCP/IP enabled (Tools → Options → VI Server).
18. `-LabVIEWPath` selects which LabVIEW version runs the operation; it is required on macOS/Linux but optional on Windows; on Windows the default is “the version of LabVIEW that was most recently used on the machine.”
19. `-Headless` runs operations headlessly and “don’t require a LabVIEW license to execute.”
Source: NI “Running Operations Using the Command Line Interface for LabVIEW” — <https://www.ni.com/docs/en-US/bundle/labview/page/running-operations-using-the-command-line-interface-for-labview.html>

Engineering recommendation (explicitly marked as an interpretation)

Primary engine: LabVIEWCLI `CreateComparisonReport` **with** `HTMLSingleFile`.

This is the only option among the documented tools that is explicitly designed for programmatic report generation and yields a single-file artifact that is easy to embed into a VS Code webview. Choosing

`HTMLSingleFile` also avoids multi-file asset path rewriting and reduces webview local resource exposure.

Fallback engine: LVCompare as interactive launcher only.

LVCompare is useful if the user wants the native comparison UI, but it does not provide authoritative documentation for headless HTML report generation; also it has same-name limitations and requires careful temp-copy/rename handling.

Required preflight checks before running either engine

These checks are a mix of **source-backed** requirements and **defensive engineering**:

Source-backed preflights * Ensure CLI arguments are passed **with correct case** (case-sensitive). (NI CLI docs) * For LabVIEWCLI: detect whether the workspace environment is set up for CLI operations (VI Server TCP/IP enabled). If not, display instructions (cannot auto-fix safely). (NI CLI docs) * For comparisons: avoid same-name load constraints by ensuring the two input VI *filenames* differ (copy/rename). (NI Comparing VIs docs; NI LVCompare same-name limitation)

Defensive engineering preflights (assumptions/interpretations) * Ensure workspace is trusted before launching external processes (VS Code Workspace Trust model).

Ensure the two Git revisions both resolve to blobs and both pass the VI magic-byte check before invoking LabVIEW tooling.

Ensure output directory is local and writeable (NI notes local directory requirement for comparison report saving in interactive UI; unclear for CLI but safest).

* Ensure engine exists (`LabVIEWCLI` or `LVCompare` located) and is executable; if not, prompt for configuration.

Exact commands the extension should run

CreateComparisonReport (preferred)

```
# Notes:
# - Arguments are case sensitive: keep exact casing.
# - Prefer HTMLSingleFile for easier webview integration.
# - Use -c to create directory; use -o to overwrite existing.
LabVIEWCLI -OperationName CreateComparisonReport \
  -vi1 "<ABS_PATH_TO_TEMP_LEFT_VI>" \
  -vi2 "<ABS_PATH_TO_TEMP_RIGHT_VI>" \
  -reportType "HTMLSingleFile" \
  -reportPath "<ABS_OUTPUT_DIR>/<type>-report-<fullFilename>.html" \
  -c -o \
  -d \
  -Headless \
  -LabVIEWPath "<ABS_PATH_TO_LabVIEW.exe_OR_labview_binary>"
```

Authoritative syntax reference: NI CreateComparisonReport documentation (operation syntax and supported arguments) — <https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>
Headless and LabVIEWPath semantics: NI CLI docs — <https://www.ni.com/docs/en-US/bundle/labview/page/running-operations-using-the-command-line-interface-for-labview.html>

LVCompare (interactive fallback)

```
# LVCompare is documented as interactive and has same-name constraints.  
# Use two temp-copied VIs with different basenames.  
"C:\Program Files\National Instruments\Shared\LabVIEW Compare\LVCompare.exe" \  
  "<ABS_PATH_TO_TEMP_LEFT_VI>" \  
  "<ABS_PATH_TO_TEMP_RIGHT_VI>" \  
  -lvpath "<ABS_PATH_TO_LabVIEW.exe>"
```

Authoritative syntax and default install paths: NI “Configuring Source Control with LVCompare.exe” — <https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvcompareexe.html>
Same-name limitation: NI KB — <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000000YHwACAW&l=en-US>

Workspace storage and webview report integration

Source-backed facts

1. `ExtensionContext.storageUri` is explicitly intended for workspace-specific extension-private storage, and the directory “might not exist and creation is up to the extension” (parent exists). It is `undefined` when no workspace/folder is open.
Source: official `vscode.d.ts` (`ExtensionContext.storageUri` doc comment) — <https://raw.githubusercontent.com/microsoft/vscode/refs/heads/main/src/vscode-dts/vscode.d.ts>
2. VS Code recommends `storageUri` for storing large workspace-scoped files.
Source: VS Code “Common Capabilities” — <https://code.visualstudio.com/api/extension-capabilities/common-capabilities>
3. Webviews cannot access local resources directly; they must use `webview.asWebviewUri` and must restrict access with `localResourceRoots`.
4. Default accessible roots are extension install directory and active workspace.
5. `WebviewOptions.localResourceRoots` allows additional roots; should be as restrictive as possible.
Source: VS Code Webview guide (“Loading local content” and “Controlling access to local resources”) — <https://code.visualstudio.com/api/extension-guides/webview>
6. Webview CSP patterns are explicitly documented, including using `webview.cspSource` and `default-src 'none'` to disable all content by default and then re-enable only what’s needed.

Source: VS Code Webview guide ("Content security policy") — <https://code.visualstudio.com/api/extension-guides/webview>

Recommended storage layout and linking model (interpretation + concretization)

Because you must preserve the **leaf report filename** format `{type}-report-{fullFilename}.html` but still avoid collisions across directories and repo roots, use:

- `context.storageUri/reports/<repoId>/<fileId>/<type>-report-<fullFilename>.html`
- `repoId` = stable hash of `repoRoot` (e.g., SHA-256 hex truncated)
- `fileId` = stable hash of the repo-relative path (or of `repoId + relPath`)
- `fullFilename` = original basename including extensions (e.g., `foo.vi`)

This preserves the required naming but prevents collisions for identical basenames in different directories.

Expose stored HTML inside a webview safely

When building the history viewer webview:

- Include the report directory in `localResourceRoots`:
- `localResourceRoots: [context.storageUri, vscode.Uri.joinPath(context.storageUri, 'reports', repoId)]` (or narrower).
- Use `webview.asWebviewUri(reportFileUri)` to create a safe URL for the stored HTML, and link to it through:
- A link inside your UI that triggers a command that opens the report inside a dedicated "Report" webview panel, **or**
- Inline `<iframe src="...">` (only if your CSP allows it; many extensions avoid iframes for simplicity/security).

CSP guidance (source-backed): start with `default-src 'none'` and allow only local styles/scripts you ship plus local report HTML you explicitly load. Pattern example from docs:

```
img-src ${webview.cspSource} https;; script-src ${webview.cspSource}; style-src ${webview.cspSource};
```

Source: VS Code Webview CSP guide — <https://code.visualstudio.com/api/extension-guides/webview>

Showing report links only when output exists (interpretation)

Before rendering "Open report" UI actions:

- Verify existence via `vscode.workspace.fs.stat(reportUri)` (preferred for remote/virtual FS), or `fs.statSync` only when `reportUri.scheme === 'file'`.
- If missing, hide the link or display "No report generated yet".

This aligns with the UX baseline and avoids dead links.

LabVIEW 2026 Q1 runtime/tool detection

Source-backed facts

1. **Default LabVIEW install roots differ by OS and bitness**, according to NI:
2. Windows (32-bit): `C:\Program Files (x86)\National Instruments\LabVIEW <VERSION>`
3. Windows (64-bit): `C:\Program Files\National Instruments\LabVIEW <VERSION>`
4. Linux: `/usr/local/natinst/LabVIEW-<VERSION>-64`
5. macOS: `/Applications/National Instruments/LabVIEW <VERSION> 64-bit`
Source: NI "Organization of LabVIEW File Structure" — <https://www.ni.com/docs/en-US/bundle/labview/page/organization-of-labview.html>
6. **LabVIEW 2023 Q3 is the final release for macOS; starting in 2024, LabVIEW is available on Windows and Linux.**
Source: NI KB "Available Languages for LabVIEW Development System" — <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z0000019MjySAE&l=en-US>
7. **LVCompare default install locations (and that it is Pro-only per the doc page), plus the `-lvpath` behavior are documented:**
8. Windows: `C:\Program Files\National Instruments\Shared\LabVIEW Compare\LVCompare.exe`
9. macOS: `/Library/Application Support/National Instruments/LabVIEW Compare/LVCompare.app/Contents/MacOS/LVCompare`
10. Linux: `/usr/local/bin/LVCompare`
11. If `-lvpath` is not specified: Windows uses the "currently registered" version; macOS uses latest version; Linux uses last installed version.
Source: NI "Configuring Source Control with LVCompare.exe" — <https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvcompareexe.html>
12. **LabVIEW CLI `-LabVIEWPath` selection and defaults are documented:**
13. Required on macOS/Linux; optional on Windows.
14. Windows default: most recently used LabVIEW version.
Source: NI "Running Operations Using the Command Line Interface for LabVIEW" — <https://www.ni.com/docs/en-US/bundle/labview/page/running-operations-using-the-command-line-interface-for-labview.html>
15. **NI has an official download page for "NI LabVIEW Command Line Interface" listing 2026 Q1 versions and indicating it includes "32-bit and 64-bit" and "Supported OS: Windows".**
Source: NI software download page — <https://www.ni.com/en/support/downloads/software-products/download.ni-labview-command-line-interface.html>

16. **NI also has an official support article describing use of NI LabVIEW CLI on Linux** (and the need for Xvfb for GUI-requiring operations).

Source: NI KB "Using NI LabVIEW CLI Without a GUI on a Linux Server" — <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000001E9euCAC&l=en-US>

Important ambiguity (source-backed conflict): The NI CLI download page lists "Supported OS: Windows", while NI documentation and support articles describe CLI usage on Linux/macOS and explicitly document Linux/macOS flags. Treat OS support as partially ambiguous; implement detection accordingly and message users clearly.

Recommended detection strategy (hybrid; assumptions explicitly marked)

Detection priority (recommended): 1. **User-configured paths** (most reliable; avoids guesswork).

2. **Scan standard install roots** from NI docs: * Windows: search both Program Files roots for `National Instruments\LabVIEW * \LabVIEW.exe`.

* Linux: search `/usr/local/natinst/LabVIEW-* -64/labview` and related.

* macOS: for legacy versions only ($\leq$ 2023 Q3), search `/Applications/National Instruments/LabVIEW * 64-bit/LabVIEW.app/...` (**assumption:** exact bundle path must be verified per installed version).

3. **Best-effort registry probing on Windows (assumption; not fully documented by NI):** read uninstall entries or NI-specific keys to map installed LabVIEW versions to executable paths, then validate by filesystem checks.

Bitness selection when both 32-bit and 64-bit are installed: * Prefer the path selected by: 1. A user setting (e.g., `viHistorySuite.tooling.preferBitness: "prefer64" | "prefer32" | "preferMostRecent"`).

2. If a LabVIEW instance is currently running, prefer matching that process path (**assumption**, best-effort via process enumeration; must be trust-gated because it calls external OS tools).

3. Else default: * For `LabVIEWCLI` operations: explicitly pass `-LabVIEWPath` to remove dependence on "most recently used" default.

* For `LVCompare`: pass `-lvpath` explicitly to remove dependence on "currently registered" default.

Source-backed need for explicit `-LabVIEWPath` / `-lvpath`: NI CLI and LVCompare docs above.

Example code sketch for tool location (TypeScript; runnable with edits)

Assumption: This extension is desktop/remote-host, so Node's `fs` and `child_process` are available. If web extension support is later required, this entire module must be reworked (web extensions cannot spawn processes).

```
// src/tooling/labviewLocator.ts
import * as vscode from 'vscode';
import * as path from 'path';
import { promises as fsp } from 'fs';

export type LabVIEWInstall = {
  labviewPath: string; // absolute path to LabVIEW.exe (Windows) or labview
```

```

binary (Linux)
  bitness: 32 | 64;
  versionHint?: string; // best-effort (folder name)
};

async function fileExists(p: string): Promise<boolean> {
  try { await fsp.stat(p); return true; } catch { return false; }
}

export async function findLabVIEWInstalls(): Promise<LabVIEWInstall[]> {
  const installs: LabVIEWInstall[] = [];

  if (process.platform === 'win32') {
    const roots = [
      'C:\\Program Files\\National Instruments',
      'C:\\Program Files (x86)\\National Instruments',
    ];
    for (const root of roots) {
      // Conservative scan: only immediate children matching "LabVIEW *"
      const entries = await safeReadDir(root);
      for (const name of entries) {
        if (!name.toLowerCase().startsWith('labview ')) continue;
        const exe = path.join(root, name, 'LabVIEW.exe');
        if (await fileExists(exe)) {
          installs.push({
            labviewPath: exe,
            bitness: root.includes('(x86)') ? 32 : 64,
            versionHint: name.replace(/^LabVIEW\s+/i, ''),
          });
        }
      }
    }
    return installs;
  }

  if (process.platform === 'linux') {
    const natinst = '/usr/local/natinst';
    const entries = await safeReadDir(natinst);
    for (const name of entries) {
      if (!name.startsWith('LabVIEW-')) continue;
      // NI docs: /usr/local/natinst/LabVIEW-<VERSION>-64
      const bin = path.join(natinst, name, 'labview');
      if (await fileExists(bin)) {
        installs.push({ labviewPath: bin, bitness: 64, versionHint: name });
      }
    }
    return installs;
  }
}

```

```

// macOS: LabVIEW is not supported for versions >= 2024 (NI KB);
// keep best-effort for legacy installs.
if (process.platform === 'darwin') {
    // Assumption: app bundle locations vary by version; verify on target
    systems.
    const appsRoot = '/Applications/National Instruments';
    const entries = await safeReadDir(appsRoot);
    for (const name of entries) {
        if (!name.toLowerCase().startsWith('labview ')) continue;
        // Conservative placeholder path; likely needs adjustment.
        const candidate = path.join(appsRoot, name, 'LabVIEW.app', 'Contents',
'MacOS', 'LabVIEW');
        if (await fileExists(candidate)) {
            installs.push({ labviewPath: candidate, bitness: 64, versionHint:
name });
        }
    }
}

return installs;
}

async function safeReadDir(dir: string): Promise<string[]> {
    try {
        const ents = await fsp.readdir(dir, { withFileTypes: true });
        return ents.filter(e => e.isDirectory()).map(e => e.name);
    } catch {
        return [];
    }
}

```

Progress UX and trust gating

Source-backed facts

1. **Workspace Trust static declaration in `package.json` is documented**, including:
2. `capabilities.untrustedWorkspaces.supported` values: `true`, `false`, or `'limited'`.
3. `'limited'` supports `restrictedConfigurations` so that workspace-defined values are not provided in Restricted Mode.
4. VS Code provides the `isWorkspaceTrusted` context key, `workspace.isTrusted`, and `workspace.onDidGrantWorkspaceTrust`.

Source: VS Code Workspace Trust Extension Guide — <https://github.com/microsoft/vscode-docs/blob/main/api/extension-guides/workspace-trust.md>

5. **Progress API existence and usage is documented**, including `vscode.window.withProgress` and `progress.report({ message })`.

Source: VS Code 1.12 release notes (introduced progress UI for long-running operations) — https://code.visualstudio.com/updates/v1_12

Source: VS Code “Common Capabilities” (Progress API) — <https://code.visualstudio.com/api/extension-capabilities/common-capabilities>

Source: Microsoft progress sample (demonstrates withProgress + cancellation) — <https://github.com/microsoft/vscode-extension-samples/blob/main/progress-sample/README.md>

6. **Status bar items are a supported extension surface** with official UX guidelines.

Source: VS Code Status Bar UX guidelines — <https://code.visualstudio.com/api/ux-guidelines/status-bar>

(For concrete API examples, Microsoft sample code exists: <https://github.com/microsoft/vscode-extension-samples/blob/main/statusbar-sample/src/extension.ts>)

7. **Webviews support message passing** (`webview.postMessage`, `acquireVsCodeApi().postMessage`, `onDidReceiveMessage`) for live UI updates.

Source: VS Code Webview guide — <https://code.visualstudio.com/api/extension-guides/webview>

Recommended trust model for `vi-history-suite` (interpretation aligned to sources)

Because this extension will invoke external tooling (Git processes already exist; LabVIEWCLI/LVCompare will be added) and will consume workspace paths and configuration, adopt:

- `capabilities.untrustedWorkspaces.supported: "limited"`
- In Restricted Mode:
 - Allow viewing history for already-open files if you consider it safe, but **disable indexing scans and any external process launches**.
 - Hide or disable “Generate report” commands and any tool auto-detection that scans filesystem aggressively.
- Use `restrictedConfigurations` to protect:
 - Configured executable paths (LabVIEWCLI/LVCompare/LabVIEWPath).
 - Any settings that influence process arguments or output paths.

This matches the Workspace Trust guide’s warning about settings controlling code execution and recommends restricting them automatically.

Progress UX design for long-running indexing / report generation

- **Indexing:** wrap indexing in `window.withProgress({ location: ProgressLocation.Window | Notification, cancellable: true })`. Maintain counters:
 - processed files / total tracked files
 - number of candidates passing magic check
 - number of eligible VIs after commit-count check
- **Report generation:**

- Use `ProgressLocation.Notification` (so the user sees it even if switching panes) and post progress messages into the webview (e.g., “Extracting blobs”, “Validating magic bytes”, “Running LabVIEWCLI”, “Writing report”).
- Add a temporary status bar item like `$(loading~spin) VI report: 42%` (**assumption:** codicon spin usage is acceptable; codicon documentation is official).

Example TypeScript snippet: `withProgress` + **status bar** + **webview messages**

```
// src/ui/progress.ts
import * as vscode from 'vscode';

export class ProgressMux {
    private status: vscode.StatusBarItem;

    constructor() {
        this.status =
vscode.window.createStatusBarItem(vscode.StatusBarAlignment.Left, 10);
        this.status.hide();
    }

    dispose() { this.status.dispose(); }

    async runWithProgress<T>(
        title: string,
        task: (ctx: {
            report: (p: { message?: string; increment?: number; ratio?: number }) =>
void;
            isCancelled: () => boolean;
        }) => Promise<T>,
        opts?: { cancellable?: boolean; webview?: vscode.Webview }
    ): Promise<T> {
        const cancellable = opts?.cancellable ?? true;
        const tokenSource = new vscode.Cancellation_tokenSource();

        return vscode.window.withProgress<T>(
            { location: vscode.ProgressLocation.Notification, title, cancellable },
            async (progress, token) => {
                token.onCancellationRequested(() => tokenSource.cancel());

                const report = (p: { message?: string; increment?: number; ratio?:
number }) => {
                    if (typeof p.increment === 'number') progress.report({ increment:
p.increment, message: p.message });
                    else progress.report({ message: p.message });

                    // Status bar: compact + non-spammy
                    if (p.ratio !== undefined) {
```

```

        const pct = Math.max(0, Math.min(1, p.ratio));
        this.status.text = `$(loading-spin) ${title}: ${Math.round(pct *
100)}%`;
        this.status.tooltip = p.message ?? title;
        this.status.show();
    }

    // Webview: live updates
    if (opts?.webview) {
        opts.webview.postMessage({ type: 'progress', title, ...p });
    }
};

try {
    const result = await task({ report, isCancelled: () =>
tokenSource.token.isCancellationRequested });
    return result;
} finally {
    this.status.hide();
}
}
);
}
}

```

Packaging, testing, and CI

Source-backed facts

1. **Official extension publishing docs define** `vsce package` **and** `vsce publish`, **publisher registration, and security restrictions** (e.g., SVG restrictions).
Source: VS Code “Publishing Extensions” — <https://code.visualstudio.com/api/working-with-extensions/publishing-extension>
2. **Official docs explicitly specify how to install a** `.vsix` **locally, including** `code --install-extension my-extension-0.0.1.vsix`.
Source: VS Code “Publishing Extensions” — <https://code.visualstudio.com/api/working-with-extensions/publishing-extension>
Source: VS Code CLI docs — <https://code.visualstudio.com/docs/config/command-line>
3. **Official testing docs recommend** `@vscode/test-cli` **and** `@vscode/test-electron`, and show how to define `.vscode-test.js` and run tests with `vscode-test`.
Source: VS Code “Testing Extensions” — <https://code.visualstudio.com/api/working-with-extensions/testing-extension>
4. **Official CI docs provide reference GitHub Actions/Azure/GitLab patterns**, including Linux headless testing via `xvfb-run` and publishing using `VSCE_PAT`.

Source: VS Code “Continuous Integration” — <https://code.visualstudio.com/api/working-with-extensions/continuous-integration>

Recommended packaging/testing/CI plan for `vi-history-suite` (interpretation)

- **Packaging:** keep `@vscode/vsce` as devDependency and provide scripts:
 - `npm run compile`
 - `npm test`
 - `npm run package` (runs `vsce package`)
- **Test strategy:**
 - Unit tests for report-path naming, storage layout, temp file naming, tool-locator parsing (no tool invocation).
 - Integration tests using `@vscode/test-electron` that:
 - Open a real Git repo fixture workspace.
 - Mock external tool calls (either by dependency inversion or by setting an env var to fake LabVIEWCLI).
- **CI:**
 - Run `npm ci`, `npm run lint`, `npm run compile`, `npm test`.
 - On Linux CI runners, use `xvfb-run -a npm test` per CI docs.
 - For publishing, only on tags and only after tests pass.

Risks, ambiguities, and explicit assumptions

Risks and ambiguities (source-backed)

- **OS support ambiguity for LabVIEW CLI:** NI’s CLI download page lists “Supported OS: Windows” for NI LabVIEW CLI, but NI’s support content describes usage on Linux and NI’s LabVIEW manual documents Linux/macOS options such as `-LabVIEWPath` being required there. Treat Linux/macOS as “best-effort supported when installed”, and communicate clearly to users.
Sources:
 - <https://www.ni.com/en/support/downloads/software-products/download.ni-labview-command-line-interface.html>
 - <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000001E9euCAC&l=en-US>
 - <https://www.ni.com/docs/en-US/bundle/labview/page/running-operations-using-the-command-line-interface-for-labview.html>
- **Same-name VI limitation is hard and must be engineered around:** documented in NI materials for LVCompare and comparison behavior. This affects Git-revision comparisons because both revisions usually share the same basename.
Sources:
 - <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000000YHwACAW&l=en-US>
 - <https://www.ni.com/docs/en-US/bundle/labview/page/comparing-vis.html>

- `HTML` report type produces external assets (`HTML` external images/styles; `XML` external images). This complicates webview integration unless you rewrite relative paths or embed resources. Source: <https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>

Explicit assumptions (not directly confirmed by authoritative sources)

- The extension will enforce “local directory only” behavior for CLI outputs (the local-directory constraint is explicitly stated for interactive report creation, but not explicitly stated for CLI outputs).
- Windows registry keys to enumerate LabVIEW installs are not specified in NI docs used here; any registry probing is best-effort and must validate results via filesystem existence checks.
- Detecting the “currently running LabVIEW instance” is not formally documented; if implemented, treat as heuristic with user override.

Acceptance criteria for each workstream

Comparison report generation

- Given two commits that modify a content-detected VI, the extension can generate an `HTMLSingleFile comparison report` using `LabVIEWCLI -OperationName CreateComparisonReport` and place it under workspace storage.
- The generated file name **must exactly match**: `{type}-report-{fullFilename}.html` (e.g., `diff-report-foo.vi.html`).
- The engine must preflight:
 - both selected revisions exist,
 - both blobs pass VI magic-byte validation,
 - temp filenames are distinct to avoid same-name load constraints,
 - workspace is trusted before process execution.
- If `LabVIEWCLI` is missing or `CreateComparisonReport` fails, the UI must show a clear actionable error message and provide a configuration path to set tool locations.

Workspace storage and webview report integration

- Reports are stored under `context.storageUri` (workspace-scoped). If the directory does not exist, it is created; behavior is resilient to `storageUri === undefined` (no workspace open).
- Webview integration uses:
 - `webview.asWebviewUri` for all local report URIs,
 - narrow `localResourceRoots` including only necessary directories,
 - a CSP that starts with `default-src 'none'` and selectively enables required sources.
- Report links/buttons appear only when the output file exists; no dead links.

LabVIEW 2026 Q1 runtime/tool detection

- On Windows, the extension can locate:
 - candidate LabVIEW installs (32/64) via standard install roots,
 - LVCompare via documented Shared path,
 - LabVIEWCLI either via configured path or by scanning the documented Shared directory root (at minimum the folder is known via NI KB articles).
- When both 32 and 64 are installed, the extension selects bitness deterministically using:

- user config override first,
- then heuristic “running LabVIEW instance” if implemented,
- else a defined default (documented in README).
- On Linux, the extension either:
 - supports report generation when CLI + LabVIEW are installed, or
 - cleanly disables report generation with an explicit message if not available.
- On macOS, the extension explicitly documents that LabVIEW ≥ 2024 is not available and report generation is therefore not expected for 2026 Q1.

Progress UX and trust gating

- Extension declares workspace-trust behavior in `package.json` using `capabilities.untrustedWorkspaces` (supported: `'limited'` or `false`) and includes `restrictedConfigurations` for any settings that influence external tools.
- In Restricted Mode, external tool invocation commands are hidden/disabled and cannot be executed successfully.
- Long-running tasks (indexing, report generation) show progress:
- Notification progress via `window.withProgress` with meaningful messages,
- Optional status bar spinner progress,
- Optional live progress inside the webview (via `postMessage`) when the webview is open.
- Cancellation is supported for indexing and report generation (best-effort; if external tool cannot be interrupted safely, cancellation stops after completion and prevents writing results).

Packaging, testing, and CI

- `vsce package` produces a `.vsix` in CI and locally.
- The VSIX can be installed via `code --install-extension <file>.vsix`.
- Tests run headlessly in CI using the recommended VS Code test tooling and Linux `xvfb-run` where required.
- CI publishes only on tags and only after tests pass, using `VSCE_PAT` as documented.

Direct Source List (plain URLs)

VS Code (official)

- Workspace Trust Extension Guide (capabilities, restrictedConfigurations, isWorkspaceTrusted):
<https://github.com/microsoft/vscode-docs/blob/main/api/extension-guides/workspace-trust.md>
- Webview guide (asWebviewUri, localResourceRoots, CSP, postMessage):
<https://code.visualstudio.com/api/extension-guides/webview>
- Common Capabilities (storageUri guidance; progress overview):
<https://code.visualstudio.com/api/extension-capabilities/common-capabilities>
- Publishing Extensions (vsce package/publish, vsix install, PAT/publisher):
<https://code.visualstudio.com/api/working-with-extensions/publishing-extension>
- Testing Extensions (@vscode/test-electron / @vscode/test-cli):
<https://code.visualstudio.com/api/working-with-extensions/testing-extension>
- Continuous Integration (CI patterns incl. xvfb-run and VSCE_PAT):
<https://code.visualstudio.com/api/working-with-extensions/continuous-integration>

- VS Code CLI documentation (`code --install-extension` etc.):
<https://code.visualstudio.com/docs/configure/command-line>
- Status Bar UX guidelines:
<https://code.visualstudio.com/api/ux-guidelines/status-bar>
- Official API definition for `storageUri` ("directory might not exist..."):
<https://raw.githubusercontent.com/microsoft/vscode/refs/heads/main/src/vscode-dts/vscode.d.ts>
- Progress UI introduction snippet (`window.withProgress`):
https://code.visualstudio.com/updates/v1_12
- Microsoft progress sample:
<https://github.com/microsoft/vscode-extension-samples/blob/main/progress-sample/README.md>
- Microsoft status bar sample code:
<https://github.com/microsoft/vscode-extension-samples/blob/main/statusbar-sample/src/extension.ts>

NI / LabVIEW (official)

- CreateComparisonReport (Comparing VIs using LabVIEW CLI):
<https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>
- Running operations using LabVIEW CLI (`-Headless` , `-LabVIEWPath` , case sensitivity, VI Server TCP/IP requirement):
<https://www.ni.com/docs/en-US/bundle/labview/page/running-operations-using-the-command-line-interface-for-labview.html>
- LVCompare install paths and syntax; `-lvpath` defaults:
<https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvcompareexe.html>
- Comparison behavior and report formats in interactive Compare VIs flow; local path note; same-name load constraint:
<https://www.ni.com/docs/en-US/bundle/labview/page/comparing-vis.html>
- NI KB: LVCompare same-name limitation; LVCompare CLI syntax summary:
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000000YHwACAW&l=en-US>
- NI doc: standard install roots by OS and bitness:
<https://www.ni.com/docs/en-US/bundle/labview/page/organization-of-labview.html>
- NI KB: macOS support ended at LabVIEW 2023 Q3; Windows+Linux from 2024 onward:
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z0000019MjySAE&l=en-US>
- NI LabVIEW CLI download page (lists 2026 Q1; notes supported OS and bitness):
<https://www.ni.com/en/support/downloads/software-products/download.ni-labview-command-line-interface.html>
- NI KB: LabVIEW CLI on Linux without GUI (Xvfb):
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000001E9euCAC&l=en-US>
- NI KB referencing LabVIEWCLI config directory on Windows (Shared\LabVIEW CLI):
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000000gNOhCAM&l=en-US>

Git documentation (official)

- `git log` documentation (including `--follow` limitations via `log.follow` note):
<https://git-scm.com/docs/git-log>
- `git show` documentation:
<https://git-scm.com/docs/git-show>

- `git ls-files` documentation:
<https://git-scm.com/docs/git-ls-files>
 - `git ls-files` man page (details on `-z` NUL termination):
<https://www.man7.org/linux/man-pages/man1/git-ls-files.1.html>
-